Scale prose surface

Status: implemented through D184. This document defines the target model for the human programming surface and the smallest first slice. The parser is `components/core/prose-parser.fs`; the current slice is covered by checks in `development/scale/tests.fs`. D184 moved `logic` definitions out of prose into `.book` packages: prose casts books, and `logic` in a `.prose` file is `LogicOutsideBook`. See `docs/books-and-spells.typ`. The decisions taken are recorded below; the questions still deferred are recommendations, not commitments.

Why a prose surface

The current language is the IR/stack surface: a token list over a stack, with `match`, `if`, `let`, and `lambda` as internal encodings. It is precise and testable, but it is not the surface the project promises: a sentence-shaped program one abstraction layer above F#. A person should be able to write a match or a conditional the way it is spoken, with full stops and commas, and without an `end` keyword closing every block.

The prose surface is a **front-end**, not a second semantics. It parses to the existing general expression IR (`ExpressionAst.Expression`) and then follows the same lowering and evaluation path as any other program. One meaning, one interpreter; the surface only decides how the meaning is written.

The Prose sentence grammar stays the specification-sentence surface (D18). This document is the **program** surface: the same audience, different job. Both use the same punctuation discipline, and neither is a natural-language interface.

The shape

A program is a sequence of sentences. A full stop ends a sentence. A comma delimits the multiple-choice construct: it separates a `match` head from its first choice and each choice from the next, so one match can be written as one sentence. There is no `end` keyword; indentation is only for the reader.

Clause words are lowercase everywhere: `match`, `when`, `otherwise`, `then`, `close`, and `logic`. A line break does not start a new sentence: a clause on the next line is still part of the match, so it is not capitalized. Only quoted text keeps its case.

Names are lowercase words separated by spaces, exactly like the Prose sentence grammar: `is alive`, `can jump`, `player`. A surface phrase the vocabulary knows is canonicalized to that vocabulary token (`is alive` -> `isAlive`); a free multiword name is a `state of player` phrase, which the parser joins into one name with spaces. No camelCase, PascalCase, or kebab-case is written in `.prose`. A constructor is a name followed by `(`; a type after `:` is a lowercase name.

The prose form puts the choices in the same sentence:

```
match 5, when 1 then "one", when 2 then "two", otherwise "many".
```

The line form remains an alternative for long matches; each clause is its own line but still part of the same match sentence:

```
match 5.
when 1 then "one".
when 2 then "two".
otherwise "many".
```

The head names the subject once. The choices are source-ordered; the first matching choice runs. `otherwise` is the wildcard choice and must be last.

Grammar

```
program      ::= sentence+
sentence     ::= command | match "." | clause | expression "."
command      ::= ("run" | "capture") expression ("with" expression)* "."
match        ::= "match" expression "." clause+
               | "match" expression ("," choice)+ "."
nested       ::= "match" expression "." clause+ "close" "."
               | "match" expression ("," choice)+ "close"
clause       ::= "when" pattern "then" expression "."
               | "otherwise" expression "."
choice       ::= "when" pattern "then" expression
               | "otherwise" expression
expression   ::= pipeline | integer | text | atom | name | call | nested
reverse      ::= pipeline ("<|" reverse)?
pipeline     ::= expression ("|>" stage)*
stage        ::= stageHead (stageArg)*
stageHead    ::= name | call
stageArg     ::= integer | text | atom | name | call
call         ::= name "(" (argument (argument)*)? ")"
argument     ::= expression | "_"
pattern      ::= "_" | name | integer | text | atom
               | constructor | typed
constructor  ::= name "(" (pattern (pattern)*)? ")"
typed        ::= pattern ":" name
integer      ::= an optionally signed base-10 integer
text         ::= a double-quoted literal with spaces and case
atom         ::= ":" name
name         ::= a lowercase word, a spaced vocabulary phrase, or a head "of" owner
phrase
```

Arguments and constructor children are separated by whitespace inside the parentheses: `f(a b)` and `Pair(x y)`. The comma never separates them; it is the multiple-choice delimiter. A typed pattern spaces the colon (`x : player`) so it is distinct from an atom (`:player`) in the same position.

The head `match e.` opens a sentence-clause list; `match e,` opens an inline choice list. In the line form every following `when` clause is on its own line but still part of the match. In the inline form every choice is preceded by a comma and the final choice is followed by the match's full stop. `otherwise` appends a wildcard choice and closes the list; a further choice after it is refused, not reordered. A `when` or `otherwise` with no open match is refused. A match with no choices is refused.

A nested match is written with `match` in a clause body and closed by `close`. The line form writes `close.`; the inline form writes `close` where the outer sentence continues. A `close` with no open nested match is `UnexpectedClose`; a nested match that reaches the end without `close` is `ExpectedClose`.

A `logic` definition is not a prose sentence. In a `.prose` file it is `LogicOutsideBook`; the `.book` language in `docs/books-and-spells.typ` owns definitions, and prose casts a book with `cast("path" "word" args...)`.

A command sentence is the Stage 1b shell surface: `run "echo" with "hello".` parses to `Call(Word "run", ...)` and lowers to the same host word as `run("echo" "hello")` (D187).

A call `name(args)` in prose is a word call or a host word (for example `cast`). In a book definition, a call to an earlier definition lowers to a closure call; the book package owns that binding. A book definition group is recursive cannot call a later definition.

The pipeline

The operator `|>` is the F#-equivalent pipeline. It is left-associative and binds looser than a call: `x |> f(a) |> g(b)` is `(x |> f(a)) |> g(b)`. F# passes the piped value as the **last** argument:

1. `x |> f(a)` is `f(a, x)`.
2. `x |> f` is `f(x)`.
3. `x |> f(a) |> g(b)` is `g(b, f(a, x))`.

The position is independent of any language: `LastArgument` is the default and `FirstArgument` is available. F# is the compatibility case for `LastArgument`; Elixir is the compatibility case for `FirstArgument`. The `Pipeline` node records the style so a front-end can choose either.

A call argument may be `_`, a hole: `subtract(_ 10)` is an explicit partial application of one argument, not a complete call. Supplied arguments are evaluated once at creation; holes bind when the value is applied. This is not automatic currying and not a bare `f a` rule.

The reverse pipe `<|` is supported for F# package compatibility but is discouraged: `f <| x` means `f(x)`, `f a <| x` means `f(a, x)`, and it is right-associative. Prefer `|>` or a direct call.

A stage may be a call or a word followed by whitespace-separated arguments: `x |> f(a)` and `x |> f a` both elaborate to `Call(f, [a; x])`. This is a pipeline-surface convenience, not general currying; outside a pipeline, a bare `f a` is not accepted. A pipeline with no stage is `ExpectedExpression`; a stage that is not callable becomes the normal call-target lowering refusal, never a silent token splice.

Mapping to the existing IR

| Prose | `ExpressionAst.Expression` | |—|—| | `match e.` + clauses | `PatternMatch(Literal/Word/Call e, clauses)` | | `match e, when p then e, ... .` | the same `PatternMatch`, from the inline form | | `when p then e.` | `ExpressionClause { Pattern = p; Guard = None; Body = [ e ] }` | | `otherwise e.` | the clause `{ Pattern = Wildcard; Guard = None; Body = [ e ] }` | | integer | `Literal(NumberLiteral n)` | | `"not found"` | `Literal(TextLiteral "not found")` | | `:player` | `Literal(AtomLiteral "player")` | | `is alive` | `Word "isAlive"` (the vocabulary token) | | `value` | `Word "value"` (a free single-word name) | | `state of player` | `Word "state of player"` (a free `of` phrase) | | `f(a b)` | `Call(Word f, [ a; b ])` | | `cast("book" "word" 21)` | `Call(Word "cast", ...)`; lowering emits `CallWord("$book.cast#3")` | | `run "echo" with "hello".` | `Call(Word "run", ...)`; lowering emits `CallWord("$shell.run#2")` | | `_` | `PatternAst.Wildcard` | | `player` | `PatternAst.Variable("player", Fresh)` | | `some(x)` | `PatternAst.Constructor("some", [ x ])` | | `x : player` | `PatternAst.TypedPattern("player", x)` |

A clause body is one expression in the first slice. A block body and a guard are named refusals, not silent extensions; a nested match is allowed behind `close`. A logic definition belongs to a `.book` file, not to this table; the book language lowers the same expressions and is documented in `docs/books-and-spells.typ`.

The smallest first slice

The first slice proves the front-end and nothing more:

- a sentence sequence: one sentence keeps its expression, two or more form a `Block` in source order;
- patterns: wildcard, fresh variable, number, text, atom, constructor, typed;
- expressions: number, text, atom, word, and a call of a word;
- `otherwise` as the final wildcard choice;
- both forms: the inline prose form delimited by commas, and the line form with one clause per line;
- text literals with spaces and case;

- lowercase spaced vocabulary names and free names (one word or an `of` phrase);
- a nested match closed by `close`;
- no pin, repeated, shadowing, or guard patterns (D130 keeps them interpreter-only);
- no `If`, `Let`, `Lambda`, or quotation in the surface yet.

The sentence sequence is the `.prose` program shape (D145); the stack seam lowers it with `EvaluatorLowering` and runs it on the kernel `LoweredAbstraction` (D146). A hyphen is not part of a name.

The tests are parse and equivalence tests:

1. Parsing either form yields exactly the hand-built `PatternMatch` AST.
2. The inline form and the line form produce the same tree.
3. The prose path and the IR path, given the same program, produce the same `ExpressionAst` tree and the same canonical portable document.
4. Two or more sentences parse to a `Block` in source order.
5. A nested match parses to a `PatternMatch` body and is closed by `close`.
6. A spaced vocabulary phrase canonicalizes; a free name joins an `of` phrase; a hyphen is refused.
7. Text with spaces and case survives the parse.
8. Each refusal has a named `ProseError` case and a test.
9. `otherwise` must be last; a duplicate or out-of-order choice is refused.

Implementation status

The first slice is landed; D144–D149 are landed with it:

1. `components/core/prose-parser.fs` defines `ProseError`, `describeError`, and `parseProgram : VocabularyEntry list -> string -> Result<Expression, ProseError>`.
2. `components/core/Scale.Core.fsproj` compiles it after `expression-ast.fs` and the vocabulary types.
3. `development/scale/tests.fs` pins both forms, the canonical portable document, the named refusals, text with spaces, spaced names and `of` phrases, nested matches, the sentence sequence, the stack seam, and the CLI mode: thirty-three prose checks in the example suite.
4. The prose expression lowers with `EvaluatorLowering` and runs on the kernel `LoweredAbstraction`; the final stack top is the result.
5. `scale --prose <path>` reads and runs a `.prose` file; `Cli.ProseFile` selects the mode, and `Entry.runProseFile` delegates to `ProseShell.runFile`.
6. `run` and `capture` are Stage 1a host words. `components/shell-host` owns the process effects, and the prose surface itself contains no process code (D183).
7. The example suite is 1530 checks (1488 examples and 42 properties); the granular suite is 33 cases and discovery is 91 tests.

Refusals

The parser returns `Result<Expression, ProseError>`, never an exception and never a guessed clause:

Case	Meaning
`ExpectedMatch`	a `when` or `otherwise` with no open match, or a program that does not start with `match`
`EmptyMatch`	a `match` head with no choice
`ExpectedMatchSeparator`	a `match` head that ends with neither a full stop nor a comma
`DuplicateOtherwise`	a second `otherwise` choice in one match

Case	Meaning
`ClauseAfterOtherwise`	a choice after the wildcard choice
`ExpectedThen`	a `when` head without `then`
`ExpectedPeriod`	a sentence that does not end with a full stop
`ExpectedClose`	a nested match that reaches the end without `close`
`UnexpectedClose`	a `close` with no open nested match
`UnclosedParenthesis`	a call or constructor with no closing `)`
`UnexpectedComma`	a comma outside the multiple-choice delimiter positions
`UnclosedText`	a quoted text literal with no closing quote
`UnexpectedEnd`	the text ends inside a sentence or expression

Relation to the existing surfaces

- The **stack surface** stays the target of lowering and the reference for differential tests. The prose parser is a client of `ExpressionAst`, and the existing expression parser remains the canonical reader of the IR document.
- **The Prose sentence grammar** stays the specification sentence language. It is not reused here: a specification sentence projects to a property, while a prose program projects to an expression. The vocabulary canonicalization is shared.
- **Decision tables** stay a library layer. A prose `match` is not a decision table, and the parser does not grow a table form (D73, D74).

Decisions taken

1. **Multiple-choice punctuation.** The comma delimits the match construct: `match e, when p then e, ..., otherwise e.` is one sentence, with a comma before each choice. The line form (`match e.` then `when ...` clauses) is the alternative. A comma outside those positions is `UnexpectedComma`.
2. **Keywords are lowercase.** `match`, `when`, `then`, `otherwise`, and `close` are lowercase everywhere. A line break does not start a new sentence; a clause on the next line is still part of the match, so it is not capitalized. Only quoted text keeps case. Names are lowercase too.
3. **Names.** Names are lowercase words separated by spaces (the Prose sentence grammar style), canonicalized through the vocabulary; a free name is a single lowercase word or a `state of player` phrase. No camelCase, PascalCase, or kebab-case is written in `.prose`.
4. **Text with spaces.** A `.prose` text literal may contain spaces and keeps its case; `.scale` and the IR keep the one-word rule (D131).
5. **Nested match.** A nested match is allowed and closed by `close` (`close.` in the line form). Indentation is cosmetic.
6. **Program result.** A `.prose` file is a sentence sequence evaluated on one stack; the final stack value is the result. One sentence keeps its expression; two or more form a `Block` in source order.
7. **File and artifact.** A prose program is a new `.prose` file. `.scale` stays the specification document; the two never share a file.

Recommendations for the remaining questions

These are recommendations, not decisions. The practical examples in the next section should settle them before the full surface is fixed.

1. **Nested match delimiter.** `close` is chosen. The alternative is indentation-based delimitation; it is not adopted because indentation is easy to get wrong and the surface otherwise treats it as cosmetic.
2. **Argument separators.** Keep whitespace inside parentheses. The comma is the choice delimiter and never separates arguments.

Practical examples

The bootstrapping set should be small enough to review and broad enough to exercise each decision. The first file is the inline prose form:

```
match 5, when 1 then "one", when 2 then "two", otherwise "many".
```

The same match in the line form is the alternative:

```
match 5.
when 1 then "one".
when 2 then "two".
otherwise "many".
```

The D144 examples:

1. A message with spaces: `match 0, otherwise "not found".`
2. A spaced vocabulary name: `match player, when is alive then can jump, otherwise "not alive".`
3. A free single-word name: `match value, when some(x) then x, otherwise 0.`
4. A free `of` name: `match state of player, when some(x) then x, otherwise 0.`
5. A typed pattern: `match player, when x : player then x, otherwise 0.`
6. A call in a body: `match player, when some(x) then is alive(x), otherwise "gone".`
7. A two-sentence program: `match 5, otherwise 0. is alive(x).`
8. A nested match: `match x, when 0 then match y, when 1 then "one", otherwise "other" close, otherwise "zero".`
9. A nested sentence match: `match x. when 0 then match y. when 1 then "one". otherwise "other". close. otherwise "zero".`

Each example becomes an equivalence test: the `.prose` parse and the hand-built `ExpressionAst` must produce the same tree and the same portable document.

What this does not claim

This document does not implement the full surface, does not change the evaluator, and does not claim that the prose path is verified. The first slice is a parser, its sentence sequence, and its equivalence tests; the evaluator seam waits for the next step. The critical review's first recommendation is met only when the full surface, its equivalence suite, and its refusals land.